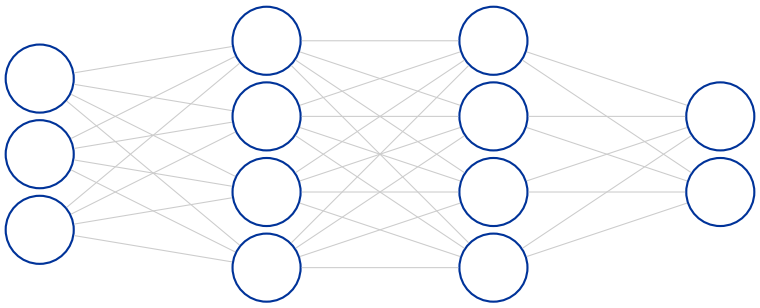
Rapport de Projet MIDL 2

Expérimentation des réseaux de neurones



Auteurs : Dalla-Costa Valentin & Elissalde Baptiste

N° Étudiants : 22407035 & 22409466

Semestre 5 – Année universitaire 2026–2027

Table des matières

1 Introduction 3

2 Description du projet 3

2.1 Contextualisation . . . 3
2.2 Notre choix d'implémentation . . . 3
2.3 Validation des gradients . . . 3
2.4 Le protocole expérimental . . . 4
2.5 Tests et résultats . . . 4

3 Annexe 5

3.0.1 Fonctions de la banque d'images . . . 5
3.0.2 Fonctions d'activation . . . 5
3.0.3 Fonction somme pondérée . . . 5
3.0.4 Fonction de coût . . . 5
3.0.5 Fonctions de création des paramètres initiaux . . . 5
3.0.6 Fonctions d'entraînement du réseau de neurones . . . 5
3.0.7 Fonctions de test du réseau de neurones . . . 5
3.0.8 Fonction main . . . 6

1 Introduction

Ce projet s'inscrit dans le cadre de l'UE PROJET MIDL 2 et porte sur l'implantation d'un réseau de neurones en python, en utilisant le plus de fonctions "fait main" que possible.

Le but de ce projet est d'implémenter une Intelligence Artificielle, c'est à dire un réseau de neurone, capable de reconnaître un chiffre sur une image.

Pour ce faire, nous avons somme tout d'abord demandé des renseignements à notre professeur d'IA, monsieur Rio Emmanuel, sur comment nous pourrions réaliser un tel projet. Ses explications nous ont offertes des pistes de réflexions afin d'avancer. Alors nous avons poursuivi nos recherches sur les réseaux de neurones et notamment d'une méthode centrale du deep learning, la descente de gradient. Ainsi après de nombreuses heure de travail à comprendre et implémenter cette méthodenous avons crée de nombreuses fonctions et avons utilisés des données judicieuses.

Ce rapport détaille donc notre démarche d'implémentation, les choix de structures de données, ainsi que l'analyse de chacune des fonctions utilisées dans notre programme.

2 Description du projet

2.1 Contextualisation

Tout d'abord, il nous parait central de définir ce qu'est l'Intelligence artificielle.
Selon Yan LeCun Professeur à l'université de New York qui a reçu le prix Turing en 2018, "... l'intelligence artificielle (IA) est un ensemble de techniques permettant à des machines d'accomplir des tâches et de résoudre des problèmes normalement réservés aux humains ...". https://www.college-de-france.fr/media/yann-lecun/UPL4485925235409209505_Intelligence_Artificielle_____Y._LeCun.pdf
Or les problèmes sont résolu par notre cerveau composés d'un réseau de neurones. Ainsi notre démarche vise a imiter ce fonctionnement. Cette méthode homonyme peut être défini comme un modèle informatique de calcul inspiré du fonctionnement du cerveau humain constituant la base du deep learning.

2.2 Notre choix d'implémentation

.....
.....
.....
.....

2.3 Validation des gradients

.....
..... Voici ce que ces chiffres signifient pour ton code et ton livrable :Une précision optimale : Les valeurs comme 2.67e-07 ou 4.16e-06 sont des

erreurs relatives infimes. Avec une perturbation $\varepsilon = 10^{-5}$, un écart de l'ordre de 10^{-6} à 10^{-8} est le meilleur résultat atteignable en raison des limites de précision des nombres à virgule flottante en informatique. Les deux méthodes trouvent donc exactement la même pente. Des formules validées : Cela confirme de manière irréfutable que les matrices dW et dB calculées par ta dérivation en chaîne (gradient analytique) sont correctes. Il n'y a aucune erreur d'indice, de transposée ou de signe dans ta fonction de rétropropagation. Un argument clé pour le rapport : Cette sortie de console valide explicitement l'exigence de la section 1.2 du sujet de projet, qui demande de "mettre en place au moins une méthode permettant de vérifier la cohérence de vos calculs". Tu peux intégrer ces résultats directement dans ton rapport (par exemple sous forme de petit tableau) pour prouver au correcteur que ton moteur d'apprentissage est fiable avant même de présenter tes taux de réussite. .....

.....

2.4 Le protocole expérimental

.....
.....
.....
.....

2.5 Tests et résultats

.....
.....
.....
.....

3 Annexe

Analyse des fonctions

Voici la liste des fonctions actuellement implantées dans le fichier (`fonctions.py`) ainsi que leur fonctionnement :

3.0.1 Fonctions de la banque d'images

- **charger_banque_apprentissage()** :
- **afficher_image_banque(index)** :
- **charger_donnees()** :
- **recuperer_valeurs_pixels(index)** :
- **recuperer_etiquette(index)** :

3.0.2 Fonctions d'activation

- **sigmoid(x)** :
- **sigmoid_derivative(x)** :
- **fonction_activation(matrix)** :

3.0.3 Fonction somme pondérée

- **somme_ponderee(A,W,B)** :

3.0.4 Fonction de coût

- **cout_mse(X, Y)** :

3.0.5 Fonctions de création des paramètres initiaux

- **initialiser_poids(taille_couches)** :
- **initialiser_biais(taille_couches)** :

3.0.6 Fonctions d'entraînement du réseau de neurones

- **entrainer_reseau_neurone(X_data, Y_data, liste_W, liste_B, alpha, taille_couches, nb_iterations)** :
- **calculer_gradiant(liste_poids, liste_biais, liste_couches, liste_Z, etiquette_vecteur)** :
- **mise_a_jour_parametres(liste_poids, liste_biais, liste_couches, delta, alpha) :** :

3.0.7 Fonctions de test du réseau de neurones

- **test_reseau_neurone_unique(image, etiquette, liste_W, liste_B)** :
- **test_global_reseau_neurone(X_data, Y_data, liste_W, liste_B)** :
- **prediction_values(prediction_vector)** :

3.0.8 Fonction main

— **main_reseau(taille_couches, alpha, nb_image_train)** :